Ejemplo de aplicación Reducción Optimización

Universidad Nacional de La Plata



Facultad de Informática

Prof. Dr. Adrian Pousa

Agenda

I. Reducción

II. Suma por reducción

   i. Problemas de la solución en GPU

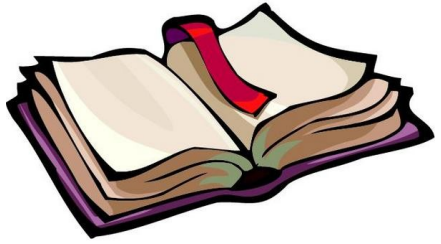
   ii. Mejoras por el uso de memoria compartida

   iii. Mejoras por divergencia

   iv. Mejoras por el patrón de acceso

   v. Mejoras adicionales

   vi. Solución final



Agenda

I. Reducción

II. Suma por reducción

   i. Problemas de la solución en GPU

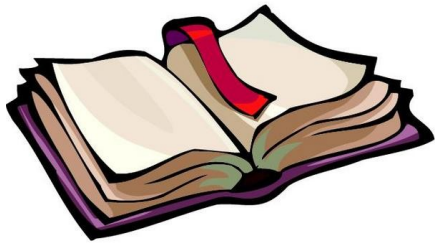
   ii. Mejoras por el uso de memoria compartida

   iii. Mejoras por divergencia

   iv. Mejoras por el patrón de acceso

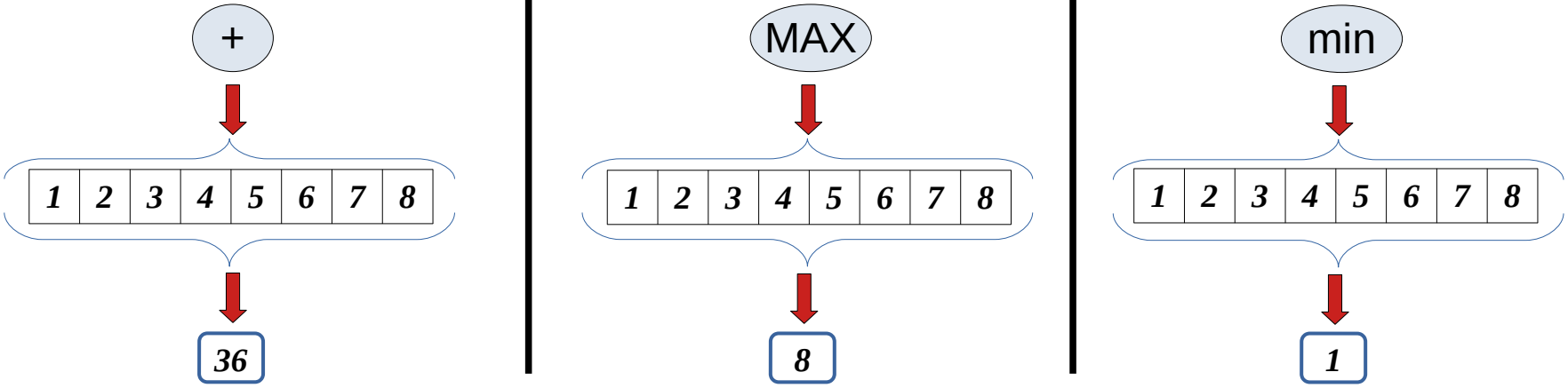
   v. Mejoras adicionales

   vi. Solución final



Reducción

- **Reducción:** reducir los valores de un vector a un valor simple.
- Es posible hacerlo aplicando un operador asociativo:
  - +, *, MAX, min, etc.



Agenda

I. Reducción

II. Suma por reducción

   i. Problemas de la solución en GPU

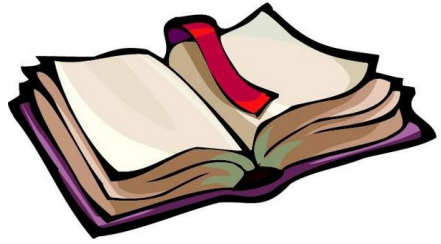
   ii. Mejoras por el uso de memoria compartida

   iii. Mejoras por divergencia

   iv. Mejoras por el patrón de acceso

   v. Mejoras adicionales

   vi. Solución final



Agenda

I. Reducción

II. Suma por reducción

   i. Problemas de la solución en GPU

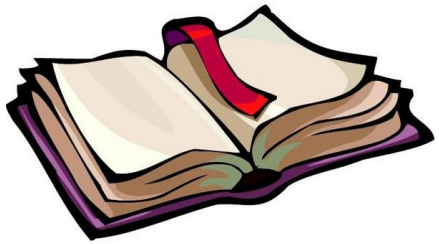
   ii. Mejoras por el uso de memoria compartida

   iii. Mejoras por divergencia

   iv. Mejoras por el patrón de acceso

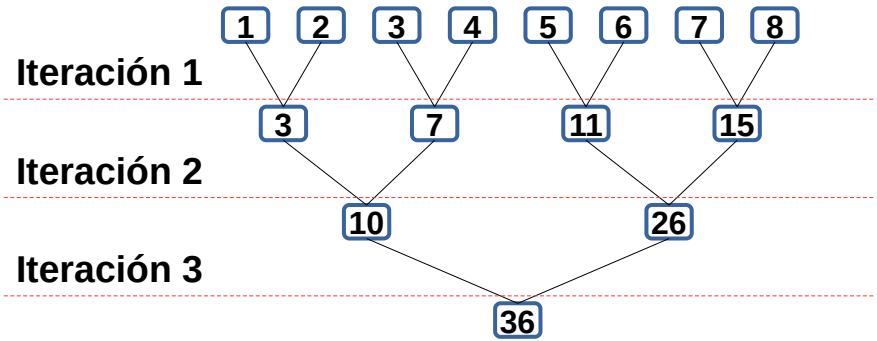
   v. Mejoras adicionales

   vi. Solución final



Suma por reducción
Problemas de la solución en GPU

- Dado un vector de N elementos, la solución de una **suma por reducción** en GPU consiste en una implementación basada en árbol.
- Se requieren varias iteraciones, en cada iteración se realiza una nueva llamada al kernel y se aprovecha el hecho de que los valores de las variables en memoria global no cambian entre llamados al kernel.
- Trabajar sobre memoria global resulta costoso.



Agenda

I. Reducción

II. Suma por reducción

   i. Problemas de la solución en GPU

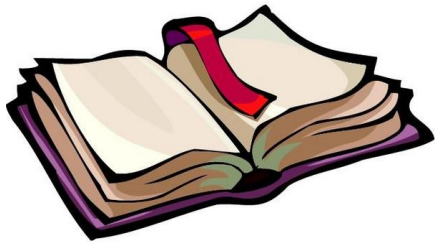
   ii. Mejoras por el uso de memoria compartida

   iii. Mejoras por divergencia

   iv. Mejoras por el patrón de acceso

   v. Mejoras adicionales

   vi. Solución final

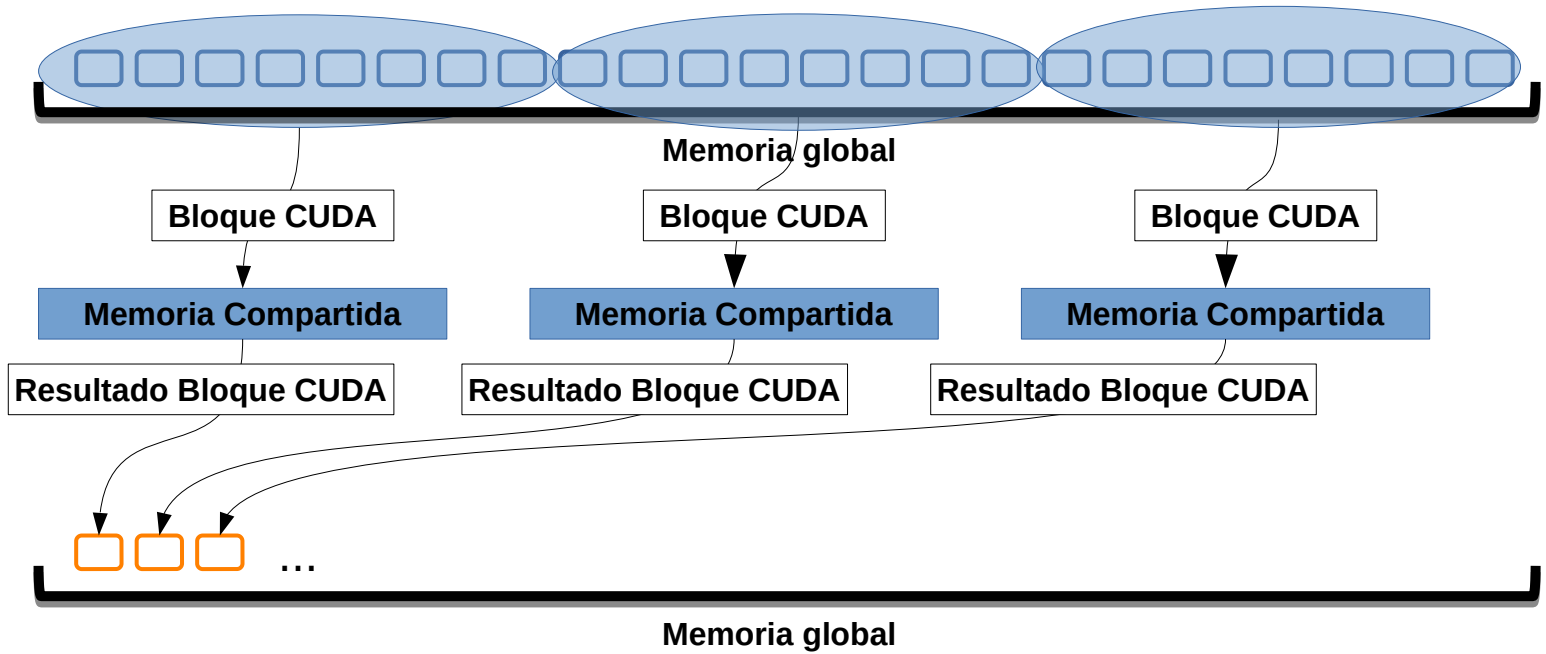


Suma por reducción
Mejoras por el uso de memoria compartida

- Podemos mejorar el tiempo de ejecución utilizando una memoria más rápida como la memoria compartida.
- La estrategia es la siguiente:
  - Cada bloque trae una porción de los datos, de forma coalescente desde memoria global a la memoria compartida.
  - Cada bloque realiza la reducción en memoria compartida.
  - Cada bloque almacena el resultado que reside en memoria compartida en la memoria global.
- Cada bloque dejará un valor. Luego, los resultados de todos los bloques deben reducirse nuevamente.

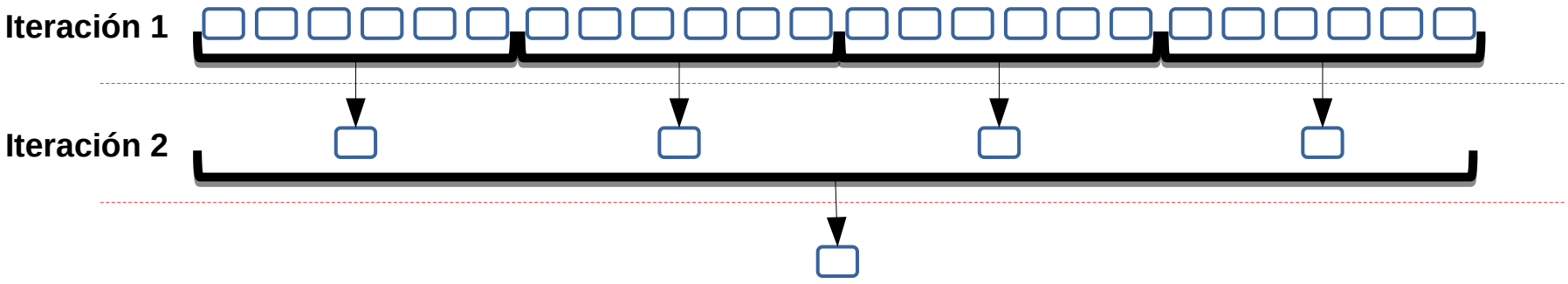
Suma por reducción
Mejoras por el uso de memoria compartida

- Una iteración consiste de:



Suma por reducción
Mejoras por el uso de memoria compartida

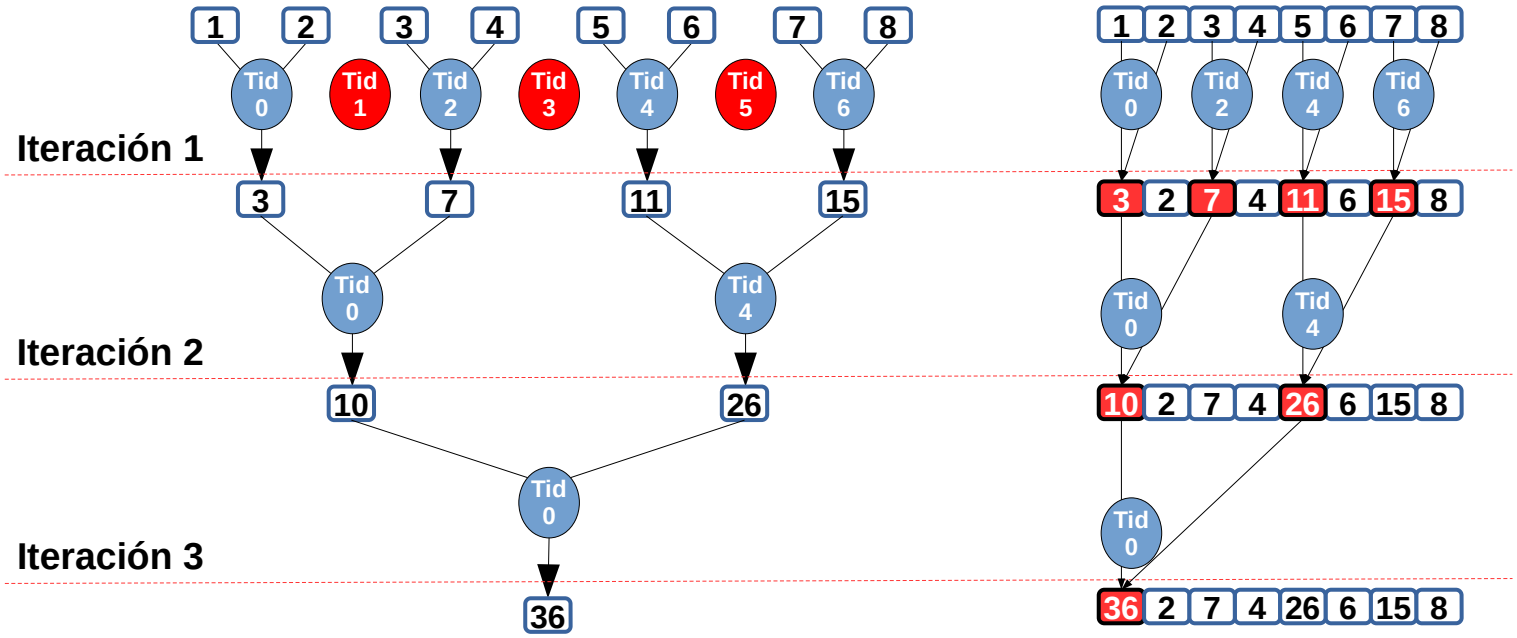
- Las siguientes iteraciones reducen los resultados de los bloques de la iteración anterior.



- A continuación analizaremos distintas soluciones dentro de cada bloque CUDA.

Suma por reducción
Solución 1 - Estrategia

- **Solución 1:** dentro de cada bloque CUDA los hilos trabajan sobre memoria compartida realizando varias iteraciones. Sólo trabajarán los hilos con `threadIdx.x` a distancia *2*iteración*.



Suma por reducción
Solución 1 - Implementación

```
__global__ void reduce(int *g_idata, int *g_odata) {
 extern __shared__ int sdata[];
 unsigned int tid = blockIdx.x*blockDim.x + threadIdx.x;

     sdata[threadIdx.x] = g_idata[tid];
     __syncthreads();

     for(unsigned int s=1; s < blockDim.x; s *= 2) {
          if (threadIdx.x % (2*s) == 0)
               sdata[threadIdx.x ] += sdata[threadIdx.x + s];

          __syncthreads();
     }

     if (threadIdx.x == 0)
           g_odata[blockIdx.x] = sdata[0];
}
```

Lectura coalescente desde memoria global a memoria compartida
Cada hilo trae un dato.

Realiza la reducción en memoria compartida

Escritura coalescente del resultado de este bloque desde memoria compartida a memoria global.

Problema: Divergencia que resulta en un bajo rendimiento.
Hilos de un mismo warp no trabajan.

Agenda

I. Reducción

II. Suma por reducción

   i. Problemas de la solución en GPU

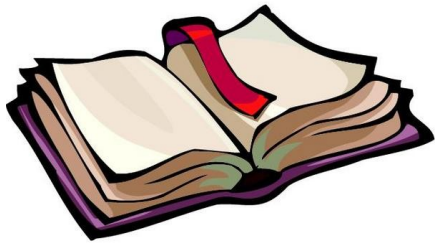
   ii. Mejoras por el uso de memoria compartida

   iii. Mejoras por divergencia

   iv. Mejoras por el patrón de acceso

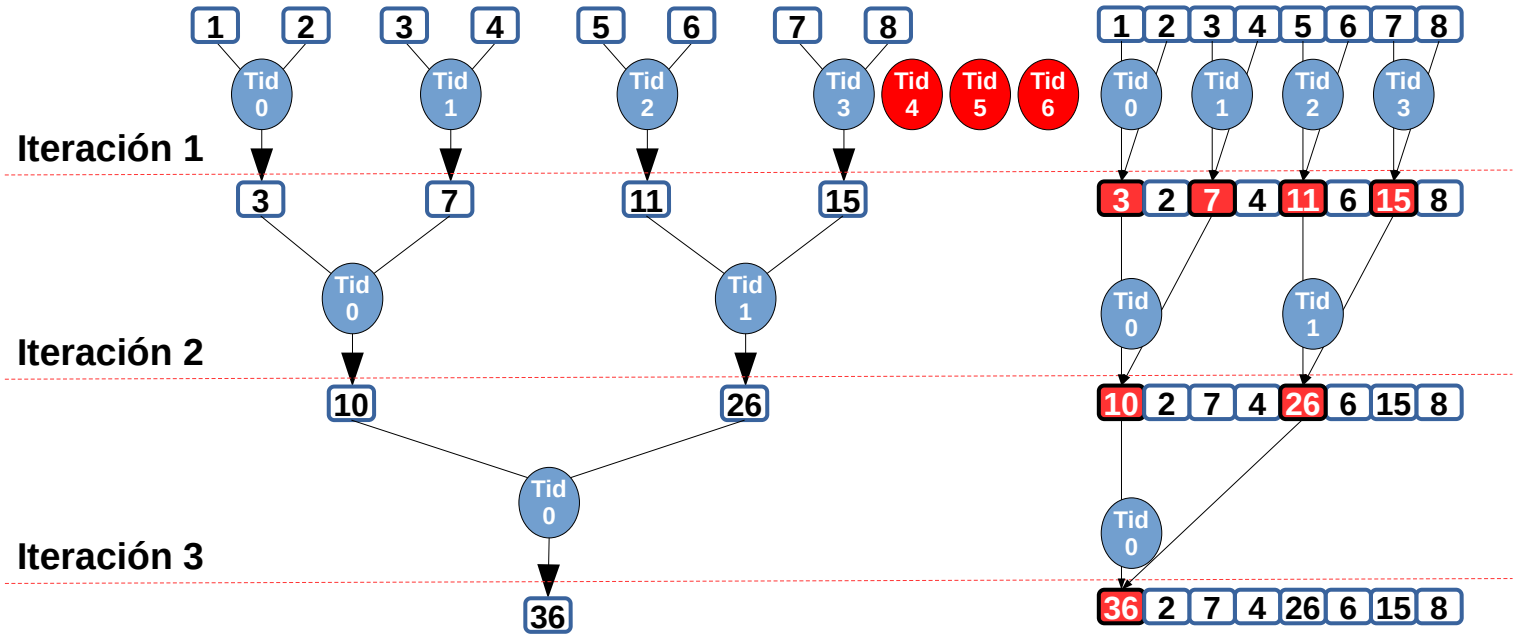
   v. Mejoras adicionales

   vi. Solución final



Suma por reducción
Solución 2 - Estrategia

- **Solución 2:** la misma estrategia reemplazando el if por otro con menor grado de divergencia.



Suma por reducción
Solución 2 - Implementación

```
__global__ void reduce(int *g_idata, int *g_odata) {
 extern __shared__ int sdata[];
 unsigned int tid = blockIdx.x*blockDim.x + threadIdx.x;

     sdata[threadIdx.x] = g_idata[tid];
     __syncthreads();

     for(unsigned int s=1; s < blockDim.x; s *= 2) {
          int index = 2 * s * threadIdx.x;
          if (index < blockDim.x)
               sdata[index] += sdata[index + s];

          __syncthreads();
     }

     if (threadIdx.x == 0)
           g_odata[blockIdx.x] = sdata[0];
}
```

Nuevo if con menor grado de divergencia

Problema: esta solución tiene conflictos de bancos en memoria compartida!!! si el vector es muy grande a medida que que se itera se accede alejadamente.

Agenda

I. Reducción

II. Suma por reducción

   i. Problemas de la solución en GPU

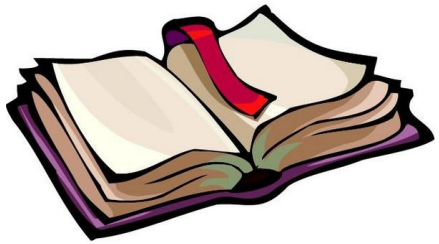
   ii. Mejoras por el uso de memoria compartida

   iii. Mejoras por divergencia

   iv. Mejoras por el patrón de acceso

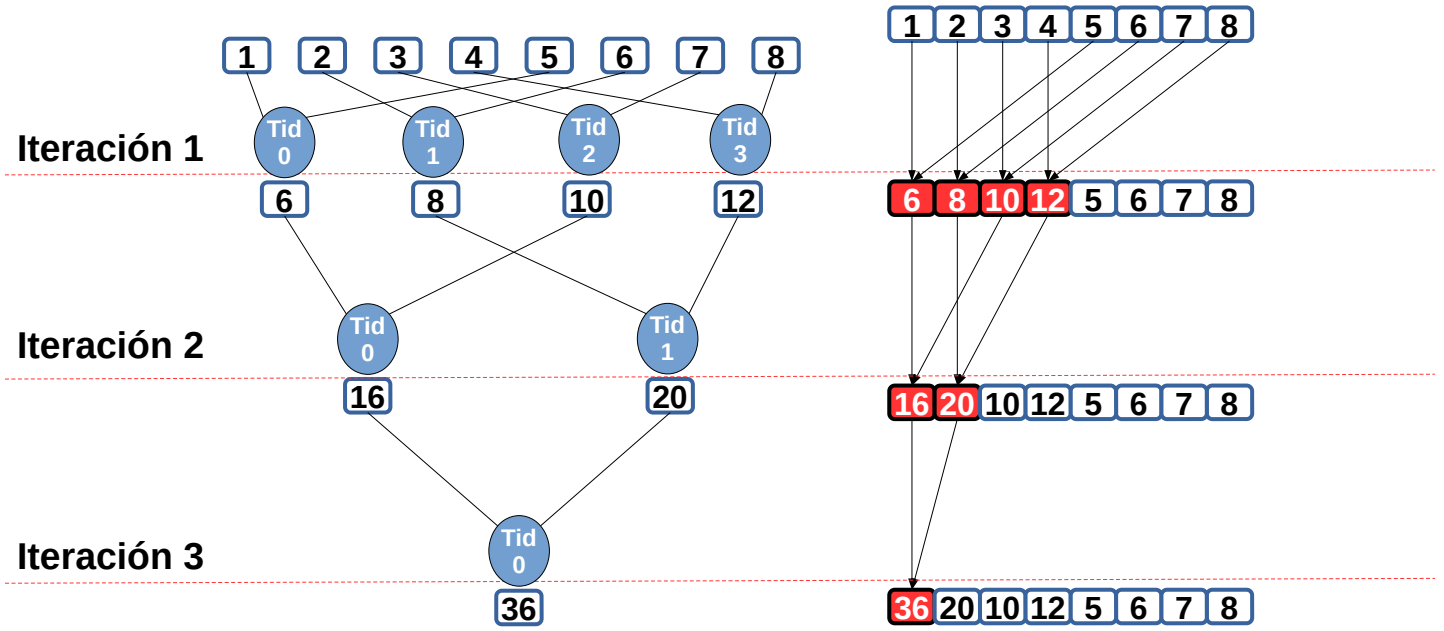
   v. Mejoras adicionales

   vi. Solución final



Suma por reducción
Solución 3 - Estrategia

- **Solución 3:** estrategia con un nuevo patrón de acceso.
- El direccionamiento secuencial está libre de conflictos.



Suma por reducción
Solución 3 - Implementación

```
__global__ void reduce(int *g_idata, int *g_odata) {
 extern __shared__ int sdata[];
  unsigned int tid = blockIdx.x*blockDim.x + threadIdx.x;

     sdata[threadIdx.x] = g_idata[tid];
     __syncthreads();

     for(unsigned int s=blockDim.x/2; s >0; s >>= 1) {
          if (threadIdx.x < s)
               sdata[threadIdx.x] += sdata[threadIdx.x + s];

          __syncthreads();
     }

     if (threadIdx.x == 0)
           g_odata[blockIdx.x] = sdata[0];
}
```

Nuevo patrón de acceso

Problema: la mitad de los hilos están inactivos/ociosos en la primera iteración.

Agenda

I. Reducción

II. Suma por reducción

   i. Problemas de la solución en GPU

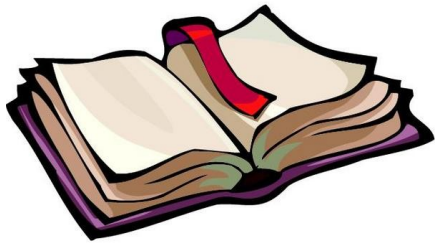
   ii. Mejoras por el uso de memoria compartida

   iii. Mejoras por divergencia

   iv. Mejoras por el patrón de acceso

   v. Mejoras adicionales

   vi. Solución final



Suma por reducción
Solución 4 – Estrategia e implementación

- **Solución 4:** se reduce a la mitad el número de bloques y se remplaza la carga simple con dos cargas de memoria global. Además se realiza la primera suma de la reducción

```
__global__ void reduce(int *g_idata, int *g_odata) {
 extern __shared__ int sdata[];
 unsigned int tid = blockIdx.x*(blockDim.x*2) + threadIdx.x;

      sdata[threadIdx.x] = g_idata[tid] + g_idata[tid + blockDim.x];
      __syncthreads();

      for(unsigned int s=blockDim.x/2; s >0; s >>= 1) {
            if (threadIdx.x < s)
                  sdata[threadIdx.x] += sdata[threadIdx.x + s];

            __syncthreads();
      }

      if (threadIdx.x == 0)
             g_odata[blockIdx.x] = sdata[0];
}
```

Suma por reducción
Solución 5 – Estrategia

- **Solución 5:** La reducción posee una baja intensidad aritmética.
- Un posible cuello de botella es el overhead de instrucciones:
  - Instrucciones auxiliares que no son Loads, Stores o Cómputo sino aritmética de direcciones y overhead del bucle.
- A medida que avanza la reducción el número de hilos activos disminuye.
  - Cuando $s <= 32$, queda sólo un warp.
  - Las instrucciones son SIMD sincrónicas dentro de un warp.Eso significa que cuando $s <= 32$:
    - No necesitamos `__syncthreads()`
    - No necesitamos "`if (tid <s)`" porque no se guarda ningún trabajo
  - **Estrategia:** Desenrollar las últimas 6 iteraciones del for más interno.

Suma por reducción
Solución 5 – Implementación

```
...

     for(unsigned int s=blockDim.x/2; s >32; s >>= 1) {
          if (threadIdx.x < s)
               sdata[threadIdx.x] += sdata[threadIdx.x + s];

          __syncthreads();
     }
     if(threadIdx.x < 32)
          sdata[threadIdx.x] += sdata[threadIdx.x + 32];
          sdata[threadIdx.x] += sdata[threadIdx.x + 16];
          sdata[threadIdx.x] += sdata[threadIdx.x + 8];
          sdata[threadIdx.x] += sdata[threadIdx.x + 4];
          sdata[threadIdx.x] += sdata[threadIdx.x + 2];
          sdata[threadIdx.x] += sdata[threadIdx.x + 1];
     if (threadIdx.x == 0)
           g_odata[blockIdx.x] = sdata[0];
}
```

Cambio en los límites del for y Desenrrollado del último bucle

Esto ahorra trabajo inútil en todos los warps, ¡no solo en el último!
Sin desenrollar, todos los warps ejecutan cada iteración del bucle for y la instrucción if

Suma por reducción
Solución 6 – Estrategia

- **Solución 6:** Si conociéramos el número de iteraciones en tiempo de compilación, podríamos desenrollar por completo la reducción:
  - El número de hilos por bloque generalmente lo conocemos
  - Además, limitamos el número de hilos por bloque a potencias de 2
- Entonces, podemos desenrollar por completo un bucle fácilmente para un bloque de tamaño fijo, pero debe hacerse de forma genérica:

¿Cómo desenrollamos el bloque para tamaños desconocidos en tiempo de compilación?

Suma por reducción
Solución 6 – Estrategia

- Podemos hacer uso de CUDA Templates (sólo C++)
- En el encabezado del kernel se debe indicar el parámetro genérico a recibir como template:

```
...
template <unsigned int blockSize>
__global__ void reduce(int *g_idata, int *g_odata){
…
}
```

- En la llamada al kernel se especifica el tamaño de bloque como un parámetro de template:

```
int main(int argc, char* argv[] ){
...
 reduce<512><<<dimGrid, dimBlock, smemSize >>>(d_idata, d_odata);
…
}
```

Suma por reducción
Solución 6 – Implementación

```
template <unsigned int blockSize>
__global__ void reduce(int *g_idata, int *g_odata){
…
if (blockSize >= 512) {    if (tid < 256) { sdata[threadIdx.x] += sdata[threadIdx.x + 256]; }
__syncthreads();           }
if (blockSize >= 256) {    if (tid < 128) { sdata[threadIdx.x] += sdata[threadIdx.x + 128]; }
__syncthreads();      }
if (blockSize >= 128) {    if (tid < 64) { sdata[threadIdx.x] += sdata[threadIdx.x + 64]; }
__syncthreads();      }
if (tid < 32) {
      if (blockSize >= 64) sdata[threadIdx.x] += sdata[threadIdx.x + 32];
      if (blockSize >= 32) sdata[threadIdx.x] += sdata[threadIdx.x + 16];
      if (blockSize >= 16) sdata[threadIdx.x] += sdata[threadIdx.x + 8];
      if (blockSize >= 8) sdata[threadIdx.x] += sdata[threadIdx.x + 4];
      if (blockSize >= 4) sdata[threadIdx.x] += sdata[threadIdx.x + 2];
      if (blockSize >= 2) sdata[threadIdx.x] += sdata[threadIdx.x + 1];
 }
...
}
```

El código relacionado a blockSize se evalúa en tiempo de compilación

Suma por reducción
Solución 6 – Implementación

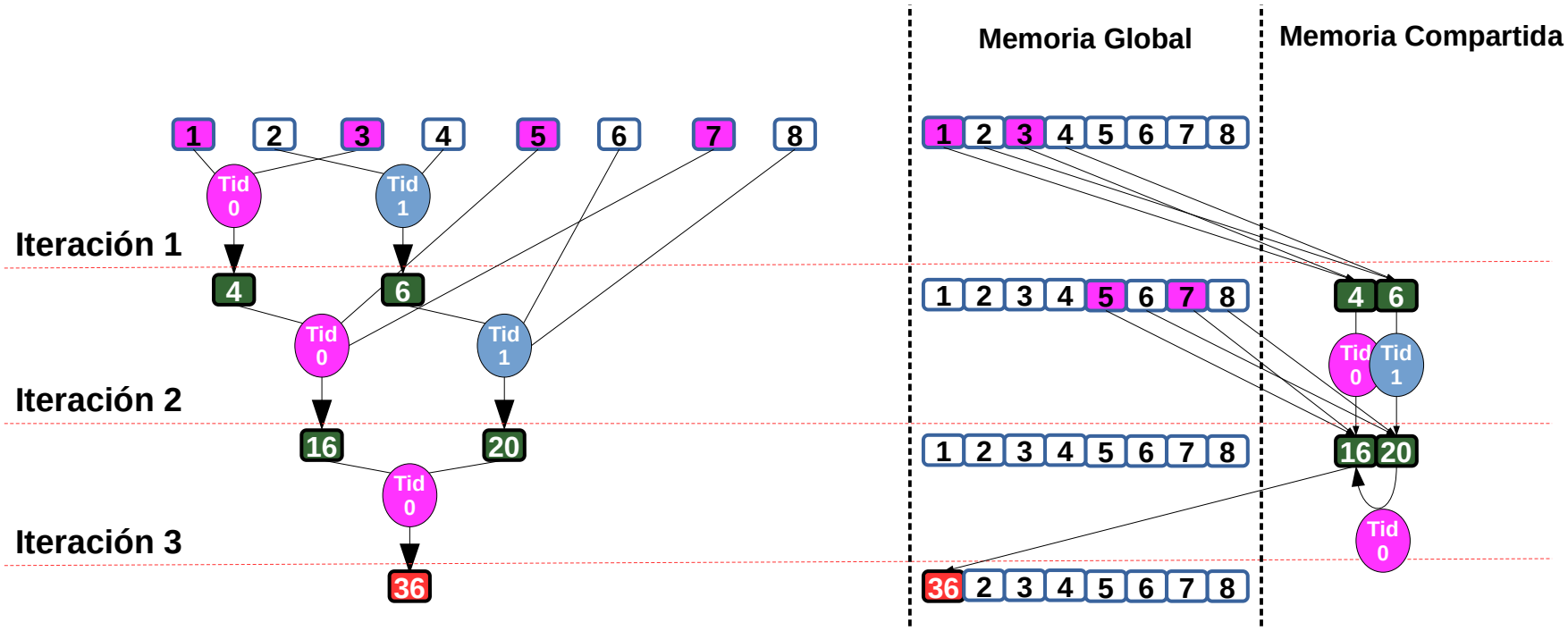
- La llamada al kernel debe especificarse para cada opción, los templates no aceptan variables:

```
int main(int argc, char* argv[]){
...
switch (threads){
 case 512: reduce<512><<< dimGrid, dimBlock, smemSize >>>(d_idata, d_odata); break;
 case 256: reduce<256><<< dimGrid, dimBlock, smemSize >>>(d_idata, d_odata); break;
 case 128: reduce<128><<< dimGrid, dimBlock, smemSize >>>(d_idata, d_odata); break;
 case 64: reduce<64><<< dimGrid, dimBlock, smemSize >>>(d_idata, d_odata); break;
 case 32: reduce<32><<< dimGrid, dimBlock, smemSize >>>(d_idata, d_odata); break;
 case 16: reduce<16><<< dimGrid, dimBlock, smemSize >>>(d_idata, d_odata); break;
 case 8: reduce< 8><<< dimGrid, dimBlock, smemSize >>>(d_idata, d_odata); break;
 case 4: reduce<4><<< dimGrid, dimBlock, smemSize >>>(d_idata, d_odata); break;
 case 2: reduce<2><<< dimGrid, dimBlock, smemSize >>>(d_idata, d_odata); break;
 case 1: reduce<1><<< dimGrid, dimBlock, smemSize >>>(d_idata, d_odata); break;
...
}
```

Suma por reducción
Solución 7 – Estrategia

- **Solución 7:** la idea es combinar una ejecución secuencial con una paralela donde cada hilo carga y suma múltiples elementos en memoria compartida.
  - Cada hilo realiza una iteración que ejecuta tantas sumas como sea necesario.
  - Cada hilo suma secuencialmente una porción de la memoria compartida.

Suma por reducción
Solución 7 – Estrategia



Suma por reducción
Solución 7 – Implementación

```
__global__ void reduce(int *g_idata, int *g_odata, unsigned int n) {
 extern __shared__ int sdata[];
 unsigned int tid = blockIdx.x*(blockSize*2) + threadIdx.x;
 unsigned int gridSize = blockSize*2*gridDim.x;
 sdata[threadIdx.x] =0;

     while(tid<n){
          sdata[threadIdx.x] += g_idata[tid] + g_idata[tid+blockSize];
          i += gridSize;
     }
     __syncthreads();
...

}
```

El loop while se mueve de a gridSize manteniendo la coalescencia.

Agenda

I. Reducción

II. Suma por reducción

   i. Problemas de la solución en GPU

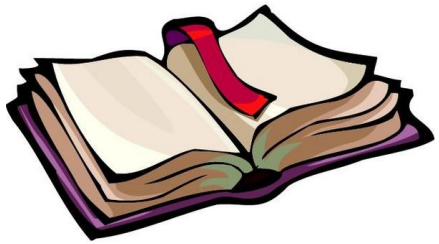
   ii. Mejoras por el uso de memoria compartida

   iii. Mejoras por divergencia

   iv. Mejoras por el patrón de acceso

   v. Mejoras adicionales

   vi. Solución final



Suma por reducción
Solución Final

- Algoritmo final:

```
template <unsigned int blockSize>
__global__ void reduce(int *g_idata, int *g_odata, unsigned int n){
 extern __shared__ int sdata[];
 unsigned int i = blockIdx.x*(blockSize*2) + threadIdx.x;
 unsigned int gridSize = blockSize*2*gridDim.x;
   sdata[threadIdx.x] = 0;
   while (i < n) {
       sdata[threadIdx.x] += g_idata[i] + g_idata[i+blockSize]; i += gridSize;
   }

   __syncthreads();

   if (blockSize >= 512) { if (threadIdx.x<256) { sdata[threadIdx.x]+=sdata[threadIdx.x + 256]; } __syncthreads(); }
   if (blockSize >= 256) { if (threadIdx.x<128) { sdata[threadIdx.x]+=sdata[threadIdx.x + 128]; } __syncthreads(); }
   if (blockSize >= 128) { if (threadIdx.x<64) { sdata[threadIdx.x]+=sdata[threadIdx.x + 64]; } __syncthreads(); }
   if (threadIdx.x < 32) {
      if (blockSize >= 64) sdata[threadIdx.x] += sdata[threadIdx.x + 32];
      if (blockSize >= 32) sdata[threadIdx.x] += sdata[threadIdx.x + 16];
      if (blockSize >= 16) sdata[threadIdx.x] += sdata[threadIdx.x + 8];
      if (blockSize >= 8) sdata[threadIdx.x] += sdata[threadIdx.x + 4];
      if (blockSize >= 4) sdata[threadIdx.x] += sdata[threadIdx.x + 2];
      if (blockSize >= 2) sdata[threadIdx.x] += sdata[threadIdx.x + 1];
   }
   if (threadIdx.x == 0) g_odata[blockIdx.x] = sdata[0];
}
```

